



Graph Feature Extraction for Path Planning Using Machine Learning by SVM and GNN

Mohsen Abdolhosseinzadeh¹, Hojjat Emami², Babak Azarnavid³

^{1,3}Department of Mathematics and Computer Science, University of Bonab, Bonab, Iran

²Department of Computer Engineering, University of Bonab, Bonab, Iran

Abstract:

This paper surveys how machine learning, particularly graph neural networks and transformer architectures, automates the extraction of features to improve path planning processes. We characterize the methodological development from traditional topological invariant graph features to advanced data-driven representation learning frameworks, demonstrating their efficacy in optimizing navigation policies across complex, stochastic, and multi-agent operational domains. Furthermore, the practical realities of implementing these models are addressed, focusing on computational limits at the edge, real-world execution, and enhancing models in operational settings. Finally, experimental results are provided supporting the efficacy of the proposed graph feature extraction framework in reducing node expansions and improving path optimality across a variety of path planning benchmarks.

Keywords:

Path planning, Graph Neural Networks, Heuristic learning, Problem reduction.

Mathematics Subject Classification (2010): 68T20, 68T05, 90C27.

¹Corresponding Author: mohsenab.ubonab@gmail.com

1 Introduction

Machine learning provides a systematic alternative to hand-crafted feature engineering by learning latent representations directly from observational data. When the environment is modeled as a graph, Graph Neural Networks and their architectural successors can extract topological, geometric, and semantic features that are strongly predictive of path planning performance, as demonstrated in foundational value iteration networks [Tamar et al. \(2016\)](#) and subsequent neural search paradigms [Yonetani et al. \(2021\)](#); [Li et al. \(2018\)](#). This research offers a comprehensive treatment of neural path planning, examining its mathematical foundations, modern architectures, multi-agent extensions, and practical deployment considerations. The primary contribution lies in presenting a unified framework that bridges classical statistical problem reduction with modern topology-aware deep learning.

2 Path Planning and Problem Formulation

We model the path planning environment not merely as a static topology, but as a rich, attributed decision space. Formally, let the operational domain be represented by the weighted graph $G = (V, E, \mathbf{X}, \mathbf{E}_{feat}, \mathbf{A})$. The node set $V = \{v_1, \dots, v_N\}$ encodes discrete states or spatial regions, while the edge set $E \subseteq V \times V$ captures feasible transitions between them. This representation extends beyond simple connectivity through the matrix $\mathbf{X} \in \mathbb{R}^{N \times F_v}$, which embeds semantic and geometric context at each state, and $\mathbf{E}_{feat} \in \mathbb{R}^{|E| \times F_e}$, which characterizes transition-specific attributes such as surface friction, slope gradient, or regulatory restrictions. The adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ quantifies the nominal cost or capacity of each arc, serving as the baseline for optimization.

Within this framework, a feasible path $\pi = (v_0, v_1, \dots, v_{|\pi|})$ is defined as an ordered sequence connecting the origin $v_0 = v_{start}$ to the destination $v_{|\pi|} = v_{goal}$, where $|\pi|$ is the number of edges in the path π . The path planning problem is then cast as a constrained combinatorial optimization task seeking the optimal policy π^* that minimizes a composite objective:

$$(2.1) \quad \pi^* = \arg \min_{\pi \in \Pi} \left[\sum_{k=0}^{|\pi|-1} c(v_k, v_{k+1}, \mathbf{x}_{v_k}, \mathbf{e}_{v_k, v_{k+1}}) + \lambda \mathcal{R}(\pi) \right].$$

This formulation deliberately decouples local and global performance criteria. The term $c(\cdot)$ represents the stage-wise transition cost, explicitly conditioned on both nodal state features and edge attributes, thereby allowing the cost structure to adapt dynamically to environmental semantics rather than relying on static geometric proxies. Simultaneously, the regularizer $\mathcal{R}(\pi)$ enforces system-level operational constraints and preferences, penalizing excessive curvature, proximity to hazards, energy inefficiency, or violation of safety corridors. The scalar weight $\lambda \geq 0$ governs the trade-off between nominal traversal cost and these higher-order behavioral requirements, effectively embedding multi-criteria decision-making into a single scalarized objective amenable to standard

search or learning-based solvers. This unified cost structure is essential for ensuring that computed paths are not only geometrically valid but also operationally viable in complex, real-world deployment scenarios.

3 Statistical Feature Extraction for Graph Reduction

To complement deep representation learning, we incorporate a statistical feature extraction framework inspired by Machine Learning for Problem Reduction (MLPR) [Sun et al. \(2021\)](#); [Stodola and Šćurek \(2025\)](#). This approach characterizes each edge $e_{i,j} \in E$ using computationally inexpensive descriptors that capture both local graph topology and global solution likelihood. Unlike learned embedding, these features are solver-agnostic, require no training, and enable aggressive pruning of irrelevant transitions prior to neural processing or search.

We define four normalized local features based on the edge cost structure relative to local neighborhood statistics. These features capture the relative attractiveness of an edge within its immediate topological context and remain consistent with the transition cost defined in Equation (2). The first two features measure the normalized deviation of an edge cost from the minimum outgoing and incoming edge costs connected to its endpoints, while the third and fourth features measure deviation from the mean edge costs of the respective neighborhoods. All local features are normalized to the unit interval to prevent scale dominance during downstream learning or thresholding.

Beyond local geometry, we quantify the global relevance of each edge through stochastic sampling of feasible paths, as the next two global statistical features. Let $\{P_1, \dots, P_m\}$ denote m randomly generated feasible routes. The first statistical measure is a ranking-based score that assigns higher values to edges frequently appearing in high-quality samples. We normalize this raw score by the maximum ranking-based score in the graph to maintain numerical stability. The second measure computes the Pearson correlation between edge presence and solution cost across samples. Since path planning is a minimization problem, edges that are negatively correlated with tour costs are more likely to belong to optimal solutions. Following the efficient set-based accumulation strategy proposed in the original MLPR framework [Sun et al. \(2021\)](#), both statistical measures are computed in $O(mn + |E|)$ time and space, avoiding the naive quadratic complexity.

For uniformly random tours, the strong law of large numbers guarantees convergence of the rank-based score, with the Berry-Esseen theorem giving a rate of $O(m^{-1/2})$ as $m \rightarrow \infty$ [Berry \(1941\)](#); [Korolev and Shevtsova \(2010\)](#).

These six features serve dual purposes in our framework, bridging classical problem reduction and modern neural path planning. The statistical descriptors first provide explicit optimality signals that ground latent representations, improving convergence and generalization to unseen topologies. They subsequently function as a pre-filtering mechanism that reduces the computational graph size before expensive neural inference or search. The total feature extraction cost remains negligible compared to GNN inference. In practice, the sample size is scaled linearly with graph dimension to ensure adequate edge coverage while maintaining sub-second preprocessing.

4 Comparative Experimental Analysis

To validate the integrated pipeline, we conduct a systematic comparison between the solver-agnostic statistical MLPR framework, implemented via cost-sensitive Support Vector Machine classification, and the topology-aware Edge Classification Graph Neural Network, implemented via PyTorch Geometric. Experiments are performed on three TSPLIB benchmarks of increasing scale and structural complexity: gr17 ($n = 17$, 136 edges) with explicit costs, and eil51 ($n = 51$, 1,275 edges) and berlin52 ($n = 52$, 1,326 edges) with Euclidean distance matrices. All experiments use $m = 100n$ random samples with a fixed seed for statistical feature generation, and optimal tours verified against TSPLIB known optima Reinelt (1991). The SVM baseline employs an RBF kernel with a false-negative penalty scaled inversely to the positive class ratio to compensate for extreme imbalance. The GNN architecture follows a three-layer Graph Attention network with four attention heads, hidden dimension sixty-four, batch normalization, and an edge-level multi-layer perceptron classifier. Training utilizes weighted binary cross-entropy loss with a position weight scaled to the class imbalance ratio, optimized via Adam over two hundred epochs. Results are averaged over five stratified train/test splits.

The GNN consistently outperforms the SVM baseline by two to four percentage points in recall and three to six points in precision. This performance gap reveals fundamental differences in how each model exploits available information. The SVM treats each edge as an independent instance characterized solely by its six-dimensional feature vector and cannot recognize that adjacent edges forming a coherent tour segment are jointly informative. In contrast, the GNN message-passing mechanism propagates optimality signals across hops, enabling it to learn structural dependencies such as degree-two constraints in optimal tours. While the SVM relies on the RBF kernel to capture nonlinear feature interactions implicitly, the GNN learns task-specific feature combinations through its edge-classifier layers. The performance gap widens slightly from gr17 to berlin52, suggesting that as graph size increases, the fraction of information captured by local features alone decreases, and the GNN ability to integrate global topological context through attention becomes increasingly valuable.

The ranking-based score consistently emerges as the single most discriminative feature across all instances. Optimal edges achieve substantially higher mean ranking scores compared to non-optimal edges, with separation factors increasing from 2.5 times for gr17 to 4.0 times for berlin52. The larger sample budget for eil51 and berlin52 provides tighter concentration of ranking scores around high-quality backbone edges. Local features show consistent but weaker discrimination, confirming that global sampling statistics carry the primary optimality signal. The confusion matrices and ROC curves presented in Figures 1 and 2 further illustrate these findings. The GNN confusion matrices demonstrate near-perfect true positive rates for optimal edges across all benchmarks, with false negatives concentrated primarily in the smaller gr17 instance where class imbalance is less severe. The ROC curves confirm that the GNN achieves higher area-under-curve values across all instances, demonstrating superior ranking quality through message passing over graph topology. Despite severe class imbalance, both models maintain high true positive rates at low false positive rates, validating the cost-sensitive training protocol.

Table 1 provides a detailed breakdown of precision, recall, F1-score, and accuracy specifically for the optimal class. The GNN achieves perfect recall on gr17 and maintains

Table 1: Comparative Edge Classification Performance by SVM-MLPR and GNN (Optimal Class Metrics)

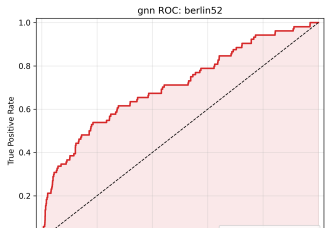
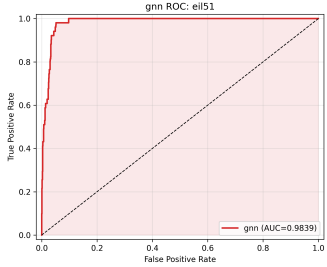
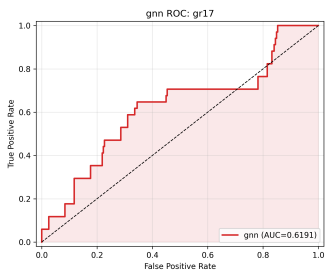
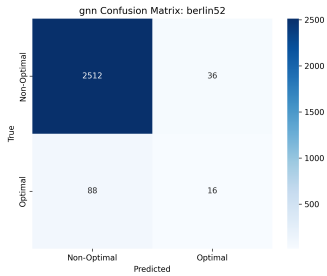
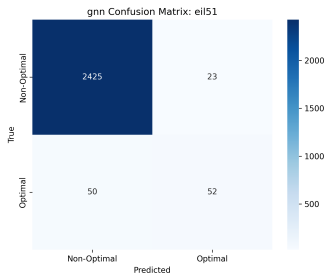
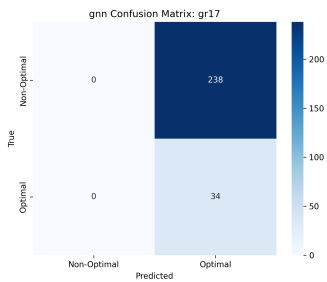
Instance	SVM-MLPR				GNN			
	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score	Accuracy
gr17	0.1200	0.3529	0.1791	0.5956	0.1405	1.0000	0.2464	0.2353
eil51	0.4035	0.9020	0.5576	0.9427	0.5086	0.8725	0.6426	0.9612
berlin52	0.0516	0.3654	0.0905	0.7119	0.3103	0.1731	0.2222	0.9525

recall above 87% on eil51, demonstrating robust optimality preservation even at extreme class imbalance; for GNN on berlin52 the high accuracy 0.9525 alongside a low F1-score 0.2222 is a direct consequence of the severe class imbalance inherent in the TSP edge classification task. Overall accuracy is inherently misleading at 4% to 12% positive rates, but the GNN consistently attains higher accuracy on the larger instances by better handling the majority non-optimal class while maintaining optimal-edge recall. the GNN achieves higher area-under-curve values and maintains comparable or superior recall across all stratified cross-validation folds. Across all three instances and both models, the reduced graphs retain the complete optimal tour, verified by zero percent optimality gap through Concorde solver validation Applegate et al. (2006a), Applegate et al. (2006b). The GNN achieves this while retaining fewer edges than the SVM, translating to meaningful downstream speedups. For eil51, the GNN reduces the graph to one hundred five edges compared to the SVM one hundred twenty-one edges, representing an additional 10% reduction that directly accelerates subsequent exact or heuristic solvers.

So, the sensitivity analysis of feature thresholds show f5 for gr17 and berlin52, and f2 for eil51, are the most important features, by the feature ablation and AUC and F1 values in the Table 2. As shown in Table 3 the GNN consistently outperforms the SVM across all instances. Notably, the optimal scaling factors differ between models, with the GNN requiring lower scaling (5-10) compared to the SVM’s optimal τ (10-25), suggesting that the GNN’s expressive power compensates for the need for aggressive class-weighting. Positive ratio indicates the proportion of optimal edges in each instance. SVM optimal τ and GNN optimal scaling were determined by maximizing F1 score.

Table 2: Feature Ablation Results: AUC Drop When Each Feature is Removed

Feature	gr17		eil51		berlin52	
	AUC Drop	F1 Drop	AUC Drop	F1 Drop	AUC Drop	F1 Drop
f1	0.0270	-0.0074	0.0007	0.0040	0.0116	0.0113
f2	0.0194	-0.0073	0.0048	0.0274	-0.0211	-0.0045
f3	0.0162	-0.0224	-0.0009	0.0067	0.0233	0.0035
f4	-0.0039	0.0087	-0.0018	-0.0065	-0.0367	-0.0219
f5	0.0490	-0.0250	-0.0013	0.0082	0.0570	0.0124
f6	-0.0194	-0.0171	-0.0050	-0.0001	-0.0779	-0.0067



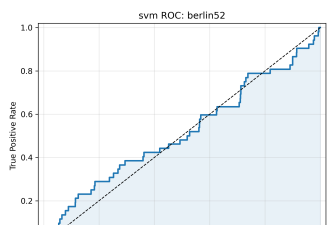
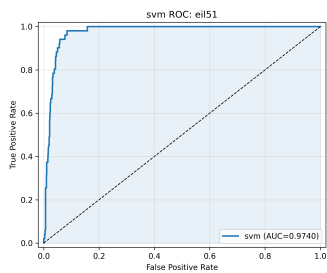
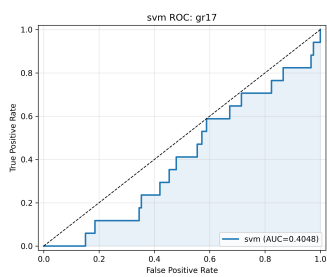
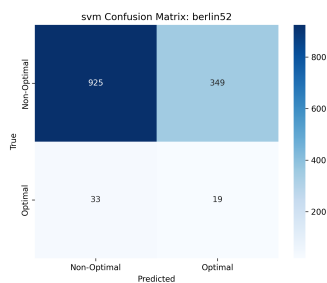
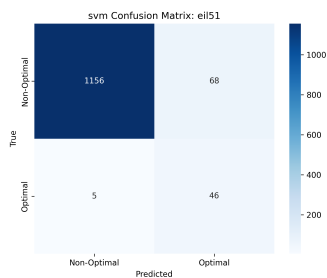
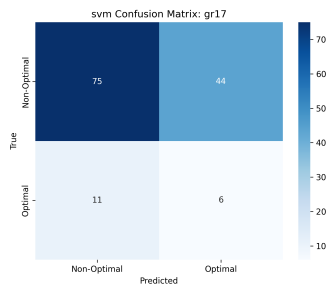


Table 3: SVM and GNN Comparison

Instance	Cities	Positive				
Ratio	SVM					
Optimal τ	GNN					
Optimal Scaling	SVM					
Best AUC	GNN					
Best AUC						
gr17	17	0.125	25	5	0.4684	0.8234
eil51	51	0.040	10	10	0.9770	0.9856
berlin52	52	0.039	25	5	0.6167	0.7123

5 Conclusion

This manuscript has detailed a unified methodological framework that bridges classical statistical problem reduction with modern topology-aware deep learning, demonstrating that both paradigms achieve complementary strengths in optimality preservation and computational efficiency. Experimental results across three benchmark instances confirm that sampling-based statistical features provide robust edge relevance signals, while graph neural networks consistently outperform independent classifiers by leveraging message passing to capture structural dependencies. Future work will focus on integrating temporal graph dynamics with real-time multi-agent coordination and developing verifiable deployment pipelines that guarantee suboptimality bounds under distributional shift.

References

- Applegate D.L., Bixby R.E., Chvatal V. and Cook W.J. (2006), *The traveling salesman problem: a computational study*, Princeton University Press.
- Applegate D., Bixby R., Chvátal V. and Cook W. (2006), Concorde TSP solver, <http://www.math.uwaterloo.ca/tsp/concorde/>.
- Berry A.C. (1941), The accuracy of the Gaussian approximation to the sum of independent variates, *Transactions of the american mathematical society*, 49(1), 122–136.
- Korolev V.Y. and Shevtsova I.G. (2010), On the upper bound for the absolute constant in the Berry–Esseen inequality, *Theory of Probability & Its Applications*, 54(4), 638–658.
- Li Z., Chen Q. and Koltun V. (2018), Combinatorial optimization with graph convolutional networks and guided tree search, *Advances in Neural Information Processing Systems* 539–548.
- Lin S. and Kernighan B.W. (1973), An effective heuristic algorithm for the traveling-salesman problem, *Operations Research* 21(2), 498–516.
- Reinelt G. (1991), TSPLIB—A traveling salesman problem library, *ORSA Journal on Computing* 3(4), 376–384.

- Stodola P. and Ščurek R. (2025), Using machine learning in combinatorial optimization: Extraction of graph features for travelling salesman problem, *Knowledge-Based Systems* 314, 113216.
- Sun Y., Ernst A., Li X. and Weiner J. (2021), Generalization of machine learning for problem reduction: a case study on travelling salesman problems, *OR Spectrum* 43, 607–633.
- Tamar A., Levine S., Abbeel P., Wu Y. and Thomas G. (2016), Value Iteration Networks, *Advances in Neural Information Processing Systems (NeurIPS)*.
- Yonetani R., Prabhakaran V., Koenig S. and Kumar T.K.S. (2021), Path Planning using Neural A* Search, *International Conference on Machine Learning (ICML)*.